

ISIT312/ISIT912 Big Data Management

HDFS Interfaces

Dr Guoxin Su and Dr Janusz R. Getta

School of Computing and Information Technology -
University of Wollongong

HDFS Interfaces

Outline

Hadoop Cluster vs. Pseudo-Distributed Hadoop

Shell Interface to HDFS

Web Interface to HDFS

Python Interface to HDFS

Internals of HDFS

Hadoop Cluster vs. Pseudo-Distributed Hadoop

A **Hadoop** cluster is deployed in a cluster of computer nodes

- As **Hadoop** is developed in Java, all **Hadoop** services sit on Java Virtual Machines running on the cluster nodes

Hadoop provides a pseudo-distributed mode on a single machine

- All Java Virtual Machines for necessary **Hadoop** services are running on a single machine
- In our case this machine is a Virtual Machine running under Ubuntu 14.04 or 22.04

HDFS provides the following interfaces to read, write, interrogate, and manage the filesystem

- The file system shell (Command-Line Interface): **hadoop fs** or **hdfs dfs**
- **Hadoop** Filesystem Java API
- **Hadoop** simple Web User Interface
- **Snakebite** pure Python HDFS client library

HDFS Interfaces

Outline

[Hadoop Cluster vs. Pseudo-Distributed Hadoop](#)

[Shell Interface to HDFS](#)

[Web Interface to HDFS](#)

[Python Interface to HDFS](#)

[Internals of HDFS](#)

Shell Interface to HDFS

Commands are provided in the shell Bash

Moving to Hadoop's home directory

```
$ cd $HADOOP_HOME
```

Home of Hadoop

You will mostly use programs in the **bin** and shell scripts in **sbin** folders, and use **jar** files in the **share** folder

Starting all Hadoop processes

```
$ ./sbin/start-all.sh
```

Starting all processes

Hadoop is running properly only if the above services are running

```
$ jps
28530 SecondaryNameNode
11188 NodeManager
28133 NameNode
28311 DataNode
10845 ResourceManager
3542 Jps
```

Listing Hadoop processes

Shell Interface to HDFS

Create a **HDFS** user account (already created in a virtual machine used by us)

```
$ bin/hdfs dfs -mkdir -p /user/bigdata
```

Creating home of user account

Create a folder **input** in a home folder

```
$ hdfs dfs -mkdir input
```

Creating a folder

View the folders in **Hadoop** home

```
$ hdfs dfs -ls
```

Listing home of user account

Found 1 item

```
drwxr-xr-x - bigdata supergroup 0 2017-07-17 16:33 input
```

What is a path to **Hadoop** home folder ?

```
$ hdfs dfs -ls /user/bigdata
```

Listing home of user account

Found 1 item

```
drwxr-xr-x - bigdata supergroup 0 2017-07-17 16:33 /user/bigdata/input
```

Shell Interface to HDFS

Upload a file to **HDFS** to **input** folder

```
$ hdfs dfs -put README.txt input
```

Uploading a file

Check the contents of **input** folder

```
$ hdfs dfs -put README.txt input
```

Uploading a file

```
$ bin/hdfs dfs -ls input
```

```
-rw-r--r-- 1 bigdata supergroup 1494 2017-07-12 17:53 input/README.txt
```

Read a file located in **HDFS**

```
$ bin/hdfs dfs -cat input/README.txt
```

Listing a file

```
< Contents of README.txt is displayed here >
```

Get help on all commands of shell interface to **HDFS**

```
$ bin/hdfs dfs -help
```

Listing a file

```
< A long list of commands of shell interface to HDFS displayed here >
```

Shell Interface to HDFS

The path in **HDFS** is represented as a URI with the prefix **hdfs://**

For example

```
hdfs://<hostname>:<port>/user/bigdata/input
```

A folder input in HDFS

refers to the **input** folder in **HDFS** under the user of **bigdata**

```
hdfs ://<hostname>:<port>/user/bigdata/input/README.txt
```

A file README.TXT in a folder input in HDFS

refers to the file **README.txt** in the above **input** directory in **HDFS**

When interacting with **HDFS** interface in the default setting, one can omitt IP, port, and user, and simply mention the directory or file

Thus, the full-spelling of **hdfs dfs -ls input** is

```
hdfs dfs -ls hdfs://<hostname>:<port>/user/bigdata/input
```


Shell Interface to HDFS

Some of frequently used commands of shell interface to [HDFS](#)

Some of the commands of HDFS shell interface

Command	Description
-put	Upload a file (or files) from the local filesystem to HDFS
-mkdir	Create a directory in HDFS
-ls	List the files in a directory in HDFS
-cat	Read the content of a file (or files) in HDFS
-copyFromLocal	Copy a file from the local filesystem to HDFS (similar to put)
-copyToLocal	Copy a file (or files) from HDFS to the local filesystem
-rm	Delete a file (or files) in HDFS
-rm -r	Delete a directory in HDFS

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

9/30

HDFS Interfaces

Outline

[Hadoop Cluster vs. Pseudo-Distributed Hadoop](#)

[Shell Interface to HDFS](#)

[Web Interface to HDFS](#)

[Python Interface to HDFS](#)

[Internals of HDFS](#)

Web Interface of HDFS

Overview 'localhost:8020' (active)

Started:	Thu Jul 27 10:53:21 AEST 2017
Version:	2.7.3, rbaa91f7c6bc9cb92be5982de4719c1c8af91ccff
Compiled:	2016-08-18T01:41Z by root from branch-2.7.3
Cluster ID:	CID-50dd23ff-1ec0-4860-b1fd-f8b0067f5b57
Block Pool ID:	BP-680435313-10.0.2.15-1499005796923

Summary

Security is off.

Safemode is off.

126 files and directories, 62 blocks = 188 total filesystem object(s).

Heap Memory used 39.84 MB of 263 MB Heap Memory. Max Heap Memory is 889 MB.

Non Heap Memory used 46.08 MB of 46.84 MB Committed Non Heap Memory. Max Non Heap Memory

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

11/30

Web Interface of HDFS

Browse Directory

/							Go!
Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	bigdata	supergroup	0 B	12/07/2017, 1:42:53 pm	0	0 B	hbase_data
drwxrwx---	bigdata	supergroup	0 B	10/07/2017, 2:23:00 pm	0	0 B	tmp
drwxr-xr-x	bigdata	supergroup	0 B	03/07/2017, 1:29:24 am	0	0 B	user

Hadoop, 2016.

HDFS Interfaces

Outline

[Hadoop Cluster vs. Pseudo-Distributed Hadoop](#)

[Shell Interface to HDFS](#)

[Web Interface to HDFS](#)

[Python Interface to HDFS](#)

[Internals of HDFS](#)

Python interface to HDFS

Snakebite is a **Python** package, created by Spotify, that allows for **HDFS** to be accessed programmatically from **Python** applications

Protocol buffer messages are used to communicate directly with the NameNode

Snakebite package also includes its own command-line interface to HDFS

A sample application of **Snakebite** that lists the contents of **/user** folder in HDFS

```
#!/usr/bin/env python3
```

```
from snakebite.client import Client
```

```
client = Client('localhost', 9000)
```

```
for x in client.ls(['/user']):  
    print(x)
```

List files

```
{'file_type': 'd', 'permission': 493, 'path': '/user/bigdata', 'length': 0, 'owner': 'bigdata',  
'group': 'supergroup', 'block_replication': 0, 'modification_time': 1711847570566, 'access_time': 0,  
'blocksize': 0}
```

Results

In HTML view press 'p' to see the lecture notes

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

14/30

Python interface to HDFS

A sample application of **Snakebite** that lists the parameters of a file **NOTICE.txt** located at **/user** in HDFS

```
#!/usr/bin/env python3

from snakebite.client import Client

client = Client('localhost', 9000)
for x in client.ls(['/user/NOTICE.txt']):
    print(x)
```

List files

```
{'file_type': 'f', 'permission': 420, 'path': '/user/NOTICE.txt', 'length': 1541, 'owner': 'bigdata',
'group': 'supergroup', 'block_replication': 1, 'modification_time': 1711864616929,
'access_time': 1711864616715, 'blocksize': 134217728}
```

List files

Python interface to HDFS

The sample applications of **Snakebite** that list the names and type of folders/files located at the end of a path

```
#!/usr/bin/env python3

from snakebite.client import Client

client = Client('localhost', 9000)
for x in client.ls(['/user/NOTICE.txt']):
    print(x['path'], x['file_type'])
```

List files

```
/user/NOTICE.txt f
```

Results

```
#!/usr/bin/env python3

from snakebite.client import Client

client = Client('localhost', 9000)
for x in client.ls(['/user/NOTICE.txt',/]):
    print(x['path'], x['file_type'])
```

List files

```
/user/NOTICE.txt f
/user d
```

Results

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

16/30

Python interface to HDFS

A sample application of **Snakebite** that copies a file from **HDFS** to a local file system

Copy file from HDFS to local file system

```
#!/usr/bin/env python3

from snakebite.client import Client

client = Client('localhost', 9000)
for x in client.copyToLocal(['/user/NOTICE.txt'], '.'):
    print(x)
```

Results

```
{'path': '/home/bigdata/NOTICE.txt', 'result': True, 'error': '', 'source_path': '/user/NOTICE.txt'}
```

ls in local file system

```
bigdata@bigdata-VirtualBox:~$ ls
```

Results

```
Desktop    Downloads  hadoop-3.3.6  NOTICE.txt  Public    snap    UMLet15-1
Documents  ex.py      Music         Pictures     shrink.sh  Templates  Videos
```

In HTML view press 'p' to see the lecture notes

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

17/30

Python interface to HDFS

A sample application of **Snakebite** that lists a file located in **HDFS**

```
#!/usr/bin/env python3

from snakebite.client import Client

client = Client('localhost', 9000)
for file in client.cat(['/user/NOTICE.txt']):
    for y in file:
        print(y)
```

List file

```
b"Apache Hadoop\nCopyright 2006 and onwards The Apache Software Foundation.\n\nThis product includes
software developed at\nThe Apache Software Foundation (https://url.au.m.mimecastprotect.com/s/2HlmC5QPyHGPEZ:
\n-----\n\nThis distribution ... "
```

Results

In HTML view press 'p' to see the lecture notes

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

18/30

Python interface to HDFS

A sample application of **Snakebite** that lists several files located in **HDFS**

```
#!/usr/bin/env python3
```

List files

```
from snakebite.client import Client

client = Client('localhost', 9000)

files = []
for x in client.ls(['/user/bigdata/NOTICE.txt']):
    if x['file_type'] == 'f':
        files.append(x['path'])

for cat in client.text(files):
    print(cat)
```

Results

```
b'\n
                                Apache License\n
                                Version 2.0, January 2004\n
https://url.au.m.mimecastprotect.com/s/Bj0mC6XQ88iK6zrgT5sph5H-n2?domain=apache.org\n\n
1. Definitions.\n\n    "License" shall mean ... "
```

Results

```
b"Apache Hadoop\nCopyright 2006 and onwards The Apache Software Foundation.\n\n
This product includes software developed at\nThe Apache Software Foundation (https://url.au.m.mimecastprotect.
\n\nExport Control Notice\n-----\n\nThis distribution includes cryptographic software. ..."
```

In HTML view press 'p' to see the lecture notes

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

19/30

Python interface to HDFS

A sample application that reads and filters a file located in [HDFS](#)

The application uses [Hadoop Streaming](#) technique to process the files located in [HDFS](#)

Assume that a file `numbers.txt` contains a sequence of numbers

```
23  
56  
78  
23  
12  
...
```

Numbers

The file is uploaded to [HDFS](#)

```
hdfs dfs -put ~/numbers.txt /user
```

Upload to HDFS

Python interface to HDFS

We would like to filter out all numbers greater than 50

```
#!/usr/bin/env python3

import sys

for number in sys.stdin:
    if int(number) > 50:
        print(int(number))
```

filter.py

The file in HDFS is processed with Hadoop Streaming feature

```
$HADOOP_HOME/bin/hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-D mapred.reduce.tasks=0 \
-input /user/numbers.txt \
-output /user/output \
-mapper ./filter.py
```

Processing

Python interface to HDFS

The results are retrieved from [HDFS](#)

```
hdfs dfs -ls /user/output
```

Looking for results

Found 2 items

```
-rw-r--r--  1 bigdata supergroup      0 2024-04-01 20:09 /user/output/_SUCCESS
-rw-r--r--  1 bigdata supergroup    8 2024-04-01 20:09 /user/output/part-00000
```

Results

```
bigdata@bigdata-VirtualBox:~/hadoop-3.3.6$ hdfs dfs -cat /user/output/part*
```

Listing results

56

78

Results

In HTML view press 'p' to see the lecture notes

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

22/30

HDFS Interfaces

Outline

[Hadoop Cluster vs. Pseudo-Distributed Hadoop](#)

[Shell Interface to HDFS](#)

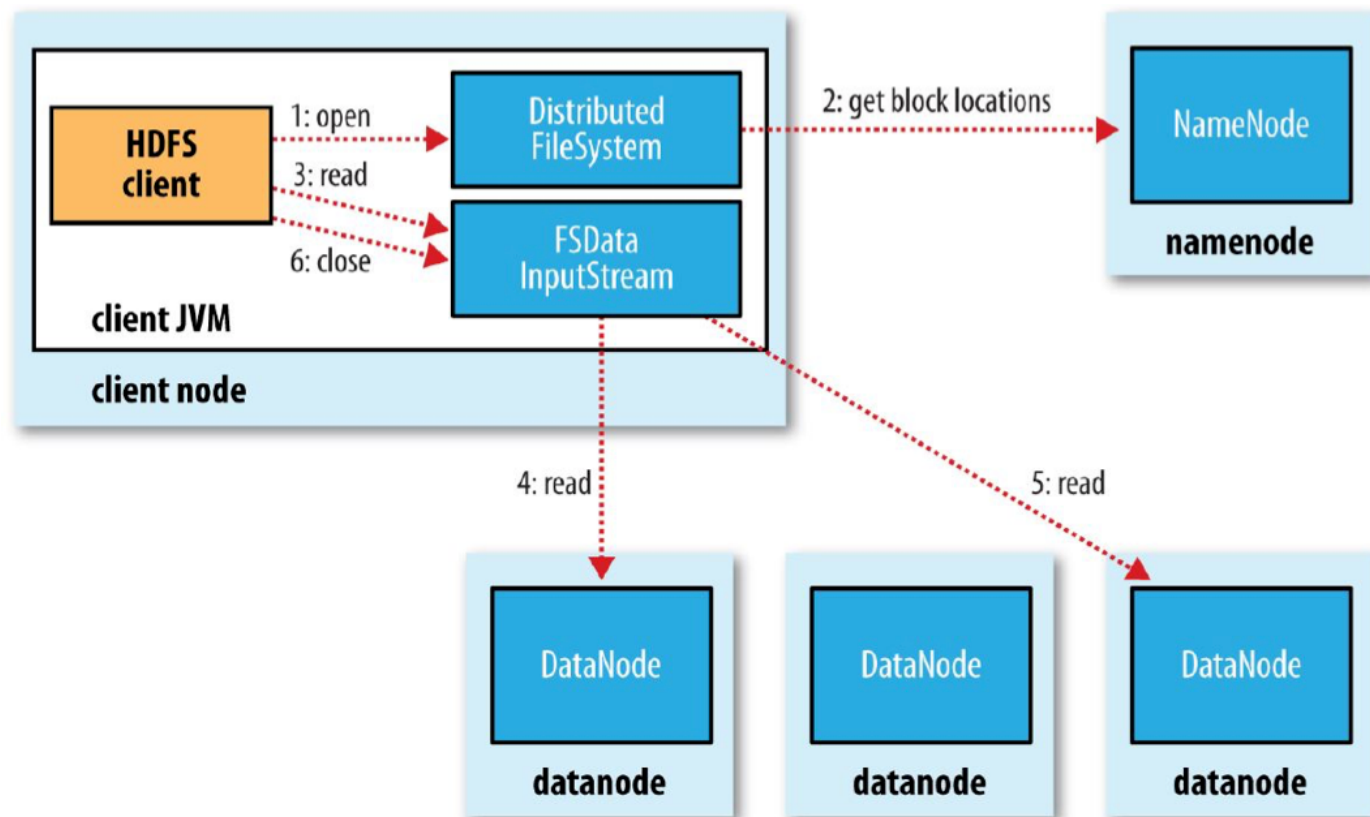
[Web Interface to HDFS](#)

[Python Interface to HDFS](#)

[Internals of HDFS](#)

Internals of HDFS

What happens "inside" when we read data into **HDFS** ?



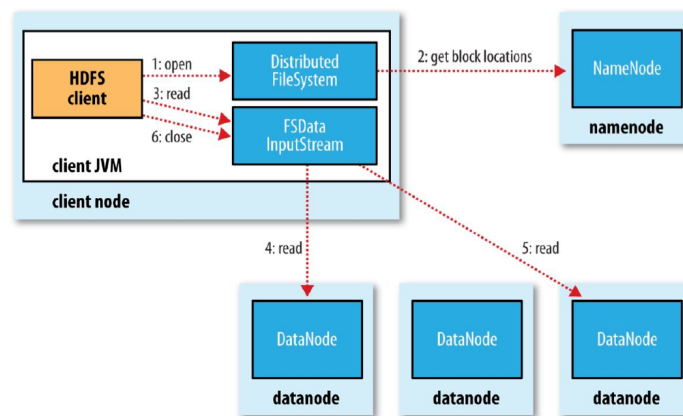
Internals of HDFS

Read data from [HDFS](#)

Step 1: The client opens the file it wishes to read by calling `open()` on the `FileSystem` object, which for [HDFS](#) is an instance of `DistributedFileSystem`

Step 2: `DistributedFileSystem` calls the `namenode`, using remote procedure calls (RPCs), to determine the locations of the first few blocks in the file

Step 3: The `DistributedFileSystem` returns an `FSDataInputStream` to the client and the client calls `read()` on the stream

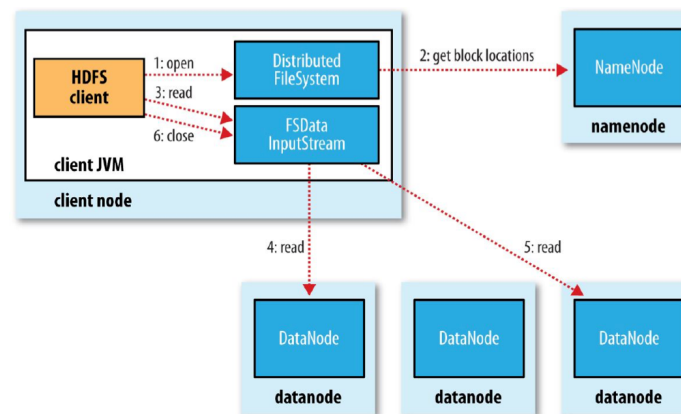


Internals of HDFS

Step 4: **FSDaataInputStream** connects to the first datanode for the first block in the file, and then data is streamed from the datanode back to the client, by calling **read()** repeatedly on the stream

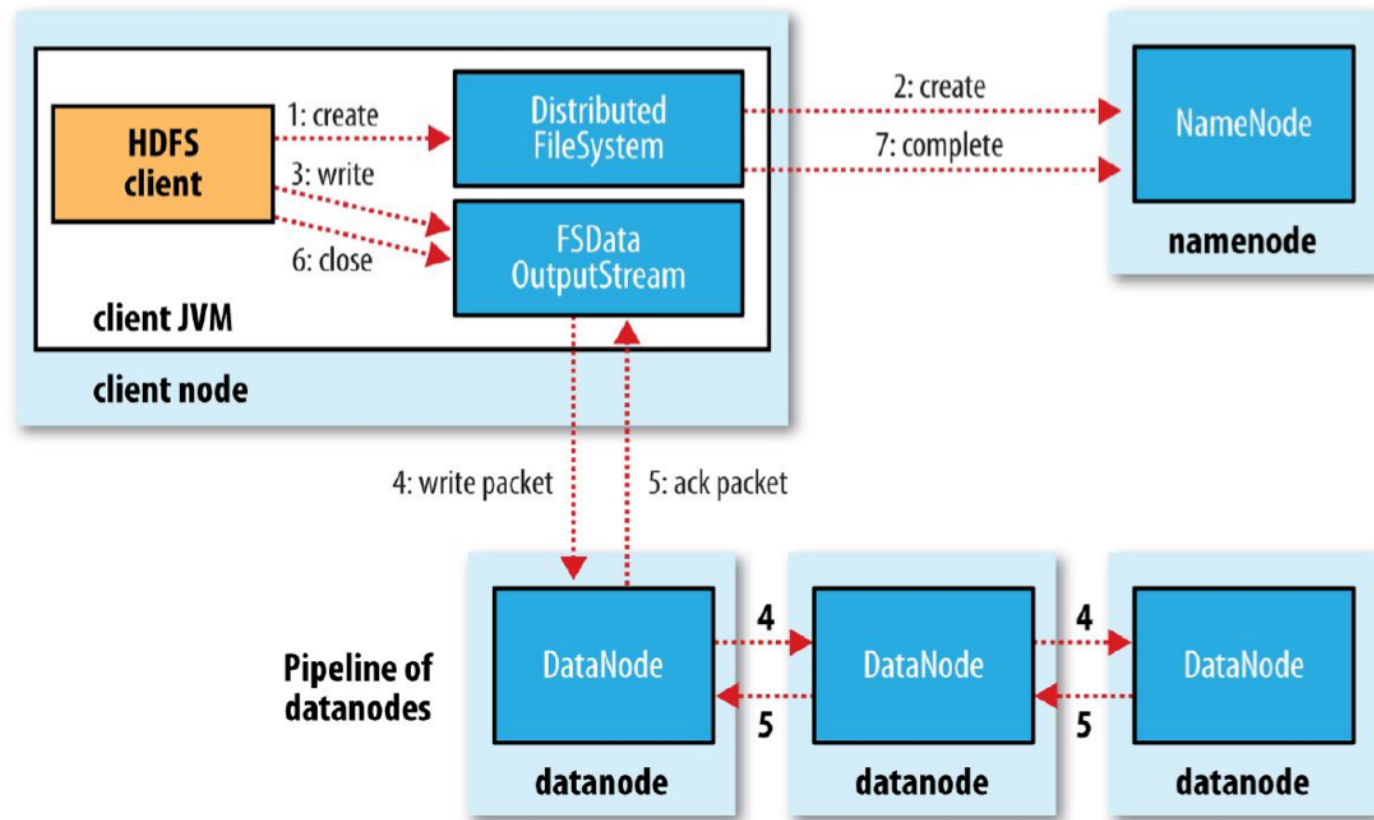
Step 5: When the end of the block is reached, **FSDaataInputStream** will close the connection to the datanode, then find the best (possibly the same) datanode for the next block

Step 6: When the client has finished reading, it calls **close()** on the **FSDaataInputStream**



Internals of HDFS

Write data into HDFS



In HTML view press 'p' to see the lecture notes
[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

27/30

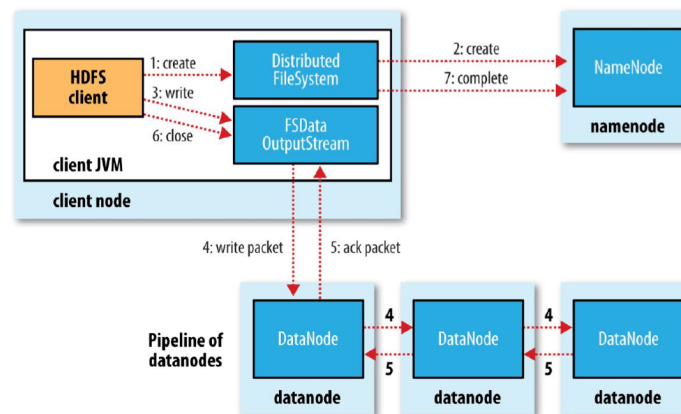
Internals of HDFS

Step 1: The client creates the file by calling `create()` on `DistributedFileSystem`

Step 2: `DistributedFileSystem` makes an RPC call to the `namenode` to create a new file in the file system namespace and returns an `FSDataOutputStream` for the client to start writing data to

Step 3: The client writes data into the `FSDataOutputStream`

Step 4: Data wrapped by the `FSDataOutputStream` is split into packages, which are flushed into a queue; data packages are sent to the blocks in a datanode and forwarded to other (usually two) datanodes

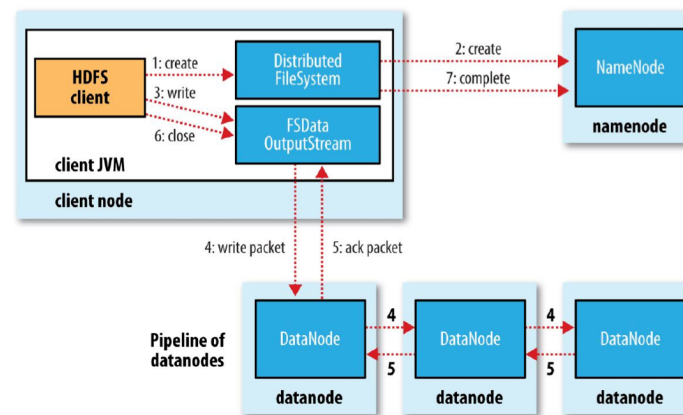


Internals of HDFS

Step 5: If `FSDataStream` receives an ack signal from the datanode the data packages are removed from the queue

Step 6: When the client has finished writing data, it calls `close()` on the stream

Step 7: The client signals the namenode that the writing is completed



References

Vohra D., Practical Hadoop ecosystem: a definitive guide to Hadoop-related frameworks and tools, Apress, 2016 (Available through UOW library)

Aven J., Hadoop in 24 Hours, SAMS Teach Yourself, SAMS 2017

[Hadoop with Python](#)

[Snakebite Documentation, Client Library](#)